Транзакции и целостность данных

Минимум для начинающих: всё или ничего в Prisma и PostgreSQL

Чему научимся

- Понять, почему несколько связанных запросов нужно выполнять вместе.
- Объяснить COMMIT и ROLLBACK на простом примере.
- Использовать prisma.$transaction([]) для заранее подготовленных запросов.
- Использовать prisma.$transaction(async (tx) => {}) для операций, зависящих друг от друга.
- Сравнить оба способа и проверить откат на примере магазина.

Проблема: одна задача — несколько запросов

- Один запрос изменяет одну запись, но задача может требовать изменения нескольких.
- Если второй запрос завершится ошибкой, результат первого без общей транзакции уже может сохраниться.
- Цель: не оставлять в базе половину одной бизнес-операции.

Транзакция простыми словами

- Транзакция — несколько действий с БД как единое целое.
- Если всё прошло успешно — COMMIT: изменения фиксируются.
- Если произошла ошибка — ROLLBACK: изменения внутри транзакции отменяются.
- Правило: либо все изменения, либо ни одного.

Пример: перевод 1 000 ₸

Состояние	Счёт А	Счёт Б	Всего
До перевода	5 000 ₸	2 000 ₸	7 000 ₸
Успешный перевод	4 000 ₸	3 000 ₸	7 000 ₸
Ошибка после списания без транзакции	4 000 ₸	2 000 ₸	6 000 ₸
Ошибка внутри транзакции	5 000 ₸	2 000 ₸	7 000 ₸

Без транзакции: что пошло не так

- 1. Списать 1 000 ₸: Первый UPDATE выполнился; баланс А = 4 000 ₸.
- 2. Зачислить 1 000 ₸: Второй UPDATE завершился ошибкой.
- 3. Последствие: Первое изменение уже сохранилось; деньги не зачислены Б.

С транзакцией: та же ошибка

- 1. Начать транзакцию: Оба изменения входят в одну операцию.
- 2. Списать 1 000 ₸: Изменение выполнено внутри транзакции.
- 3. Ошибка зачисления: Возникает исключение, выполняется ROLLBACK.
- 4. Итог: Оба баланса остаются прежними.

COMMIT и ROLLBACK

COMMIT

- Все действия выполнены успешно.
- База фиксирует изменения.
- Итог перевода: А = 4 000, Б = 3 000.

ROLLBACK

- Внутри транзакции возникла ошибка.
- База отменяет изменения этой транзакции.
- Итог: А = 5 000, Б = 2 000.

ACID — только главная мысль

- Atomicity: всё или ничего — ключевое свойство этого урока.
- Consistency: ограничения базы не должны нарушаться.
- Isolation: параллельные транзакции учитывают друг друга по правилам изоляции.
- Durability: зафиксированные данные сохраняются.
- Сейчас работаем практически только с атомарностью.

Транзакции в Prisma

Два подхода: массив запросов и функция с tx

Два способа транзакций в Prisma

- Способ 1: $transaction([запрос1, запрос2]) — передаём массив Prisma-запросов.
- Способ 2: $transaction(async (tx) => { ... }) — передаём функцию с последовательностью действий.
- В обоих случаях Prisma фиксирует изменения при успехе и откатывает их при ошибке.
- Выбор зависит от того, нужны ли результаты и проверки между запросами.

Способ 1. Массив $transaction([])

```javascript
const [productCount, categoryCount] =
  await prisma.$transaction([
    prisma.products.count(),
    prisma.categories.count()
  ]);

console.log("Товаров:", productCount);
console.log("Категорий:", categoryCount);
```

Что важно

- Массив содержит Prisma-запросы без await внутри элементов.
- Результаты возвращаются массивом в том же порядке.
- Здесь только чтение — откат не меняет данные, но демонстрирует синтаксис.
- products и categories — имена моделей текущего учебного проекта.

Массив: две независимые операции UPDATE

```javascript
await prisma.$transaction([
  prisma.products.update({
    where: { id: 1 }, data: { price: 5000 }
  }),
  prisma.products.update({
    where: { id: 2 }, data: { price: 10000 }
  })
]);
```

Что важно

- Оба id известны заранее: результат первого запроса второму не нужен.
- Если оба товара существуют — обе цены меняются.
- Если второй update завершится ошибкой (например, id отсутствует) — откатится и первый.
- Перед опытом проверьте существование товаров и сохраните первоначальные цены.

Ограничение массива $transaction([])

- Нельзя прочитать результат первого запроса в JavaScript и подставить полученный order.id во второй элемент этого же массива.
- Внутри массива нет места для условной проверки между запросами.
- Если данные следующего запроса зависят от результата предыдущего, удобнее использовать функцию с tx.
- Для создания связанных записей одним запросом существует nested write, но здесь его не изучаем.

Способ 2. Функция $transaction(async (tx) => {})

```javascript
await prisma.$transaction(async (tx) => {
  // Первое изменение через tx
  await tx.products.update({
    where: { id: 1 },
    data: { price: 100000 }
  });

  // Второе изменение через tx
  await tx.products.update({
    where: { id: 2 },
    data: { price: 200000 }
  });
});
```

Что важно

- tx — специальный клиент для запросов, которые должны выполниться в рамках одной транзакции.
- Если обе операции успешны — Prisma выполняет COMMIT.
- Если возникает исключение — Prisma выполняет ROLLBACK.
- Пример требует существующих товаров id=1 и id=2.

Намеренно вызываем ошибку

```javascript
try {
  await prisma.$transaction(async (tx) => {
    await tx.products.update({
      where: { id: 1 },
      data: { price: 100000 }
    });

    throw new Error("Учебная ошибка");
  });
} catch (error) {
  console.log("Откат:", error.message);
}
```

Что важно

- Сначала выполняется UPDATE внутри транзакции.
- throw прерывает callback.
- ROLLBACK отменяет изменение цены.
- Перед запуском убедитесь, что товар id=1 существует.

Функция tx: результат первого запроса нужен второму

```javascript
await prisma.$transaction(async (tx) => {
  const order = await tx.order.create({
    data: { status: "new" }
  });

  await tx.orderItem.create({
    data: { orderId: order.id, productId: 1, quantity: 2 }
  });
});
```

Что важно

- Первая операция возвращает созданный заказ и его id.
- Вторая операция использует order.id — поэтому подходит форма с tx.
- Обе операции выполняются внутри одной транзакции.
- Это схема примера: для запуска нужны модели order/orderItem и их обязательные поля.

Сравнение двух способов

Вопрос	Массив $transaction([])	Функция $transaction(async (tx) => {})
Что передаём	Массив Prisma-запросов	Асинхронную функцию
Нужны await между шагами	Нет	Да
Использовать результат первого шага	Не в отдельном JS-шаге	Да, через переменную
Проверки и условия между шагами	Нет	Да
Ошибка на любом шаге	ROLLBACK	ROLLBACK
Типичный пример	Два UPDATE по известным id	Order → OrderItem

Как выбрать подход

- Два независимых UPDATE по известным id → удобен $transaction([]).
- Создать Order и затем использовать order.id → удобен $transaction(async (tx) => {}).
- В обоих случаях изменения либо фиксируются вместе, либо отменяются вместе.
- Внутри callback обращайтесь к БД через tx, а не через обычный prisma.

Пример: интернет-магазин

Создать два товара одной операцией; если второй не создался, первого тоже не должно быть

Сценарий магазина

Шаг	Действие	Условие
1	Создать товар «Кабель USB»	category_id указывает на существующую категорию
2	Создать товар «Чехол»	category_id указан ошибочно: категории нет
3	Без транзакции	«Кабель USB» может остаться в базе
4	С транзакцией	Обе операции отменяются

Магазин: две записи в одной транзакции

```javascript
await prisma.$transaction(async (tx) => {
  await tx.products.create({
    data: {
      name: "Кабель USB — demo",
      price: 2500,
      in_stock: true,
      category_id: 2
    }
  });

  await tx.products.create({
    data: {
      name: "Чехол — demo",
      price: 3900,
      in_stock: true,
      category_id: 999999
    }
  });
});
```

Что важно

- Ожидается, что категория id=2 существует.
- Предполагается, что категории id=999999 не существует, а внешний ключ действительно настроен.
- Ошибка второго INSERT отменяет также первый INSERT.
- Имена моделей здесь соответствуют учебной БД: products, categories.

Как проверить ROLLBACK на практике

- До: SELECT по тестовым именам: записей нет.
- Запуск: Запустите пример с заведомо неверной category_id.
- Ошибка: Второй запрос нарушает внешний ключ; транзакция откатывается.
- После: Проверьте: ни «Кабель USB — demo», ни «Чехол — demo» не появились.

Что такое целостность данных

- PRIMARY KEY не допускает дублирования идентификатора.
- FOREIGN KEY не даёт указать отсутствующую категорию, если связь настроена в PostgreSQL.
- NOT NULL запрещает пустое значение обязательного поля.
- Транзакция не заменяет эти ограничения — они помогают обнаружить ошибку.

Главное, что нужно запомнить

1. Транзакция объединяет изменения: COMMIT при успехе, ROLLBACK при ошибке.
2. $transaction([]) — массив заранее подготовленных Prisma-запросов.
3. $transaction(async (tx) => {}) — действия с проверками и зависимостью от предыдущих результатов.
4. В callback используем tx для всех запросов, которые должны войти в транзакцию.
5. На примере магазина проверяем, что при ошибке второго INSERT первый тоже отменён.

Контрольные вопросы

1. Что может случиться с двумя INSERT без транзакции, если второй завершится ошибкой?
2. В чём разница между COMMIT и ROLLBACK?
3. Что передаётся в $transaction([]) и в каком порядке возвращаются результаты?
4. Почему order.id удобно использовать в форме async (tx) => {}?
5. Что произойдёт, если внутри callback один запрос выполнить через prisma, а не tx?
6. Как проверить ROLLBACK в таблице products?